

Tháng 3 2026

FACADE

Trình bày bởi Nhóm 5

NỘI DUNG

01. Vấn đề cần giải quyết

02. Định nghĩa facade

03. Ý tưởng cốt lõi

04. Thành phần chính

05. UML

06. Luồng hoạt động

07. Đánh giá

08. So sánh

VẤN ĐỀ CẦN GIẢI QUYẾT

Đặt tình huống thực tế:

- Trong nhiều hệ thống, một chức năng tưởng như đơn giản ở bên ngoài lại cần phối hợp nhiều class, service hoặc subsystem bên trong.

Làm sao cung cấp một interface đơn giản, thống nhất để client sử dụng một hệ thống phức tạp bên trong.

ĐỊNH NGHĨA FACADE PATTERN

Facade Pattern là một design pattern thuộc nhóm Structural Pattern.

Nó cung cấp một lớp trung gian có interface đơn giản để che giấu sự phức tạp của một subsystem.

Nói đơn giản:

- Facade là “mặt tiền” của một hệ thống phức tạp.

Client không cần biết bên trong hệ thống có bao nhiêu class, gọi theo thứ tự nào, xử lý ra sao. Client chỉ cần gọi một phương thức cấp cao từ Facade.

Ý TƯỞNG CỐT LÕI

Ý tưởng chính của Facade Pattern là:

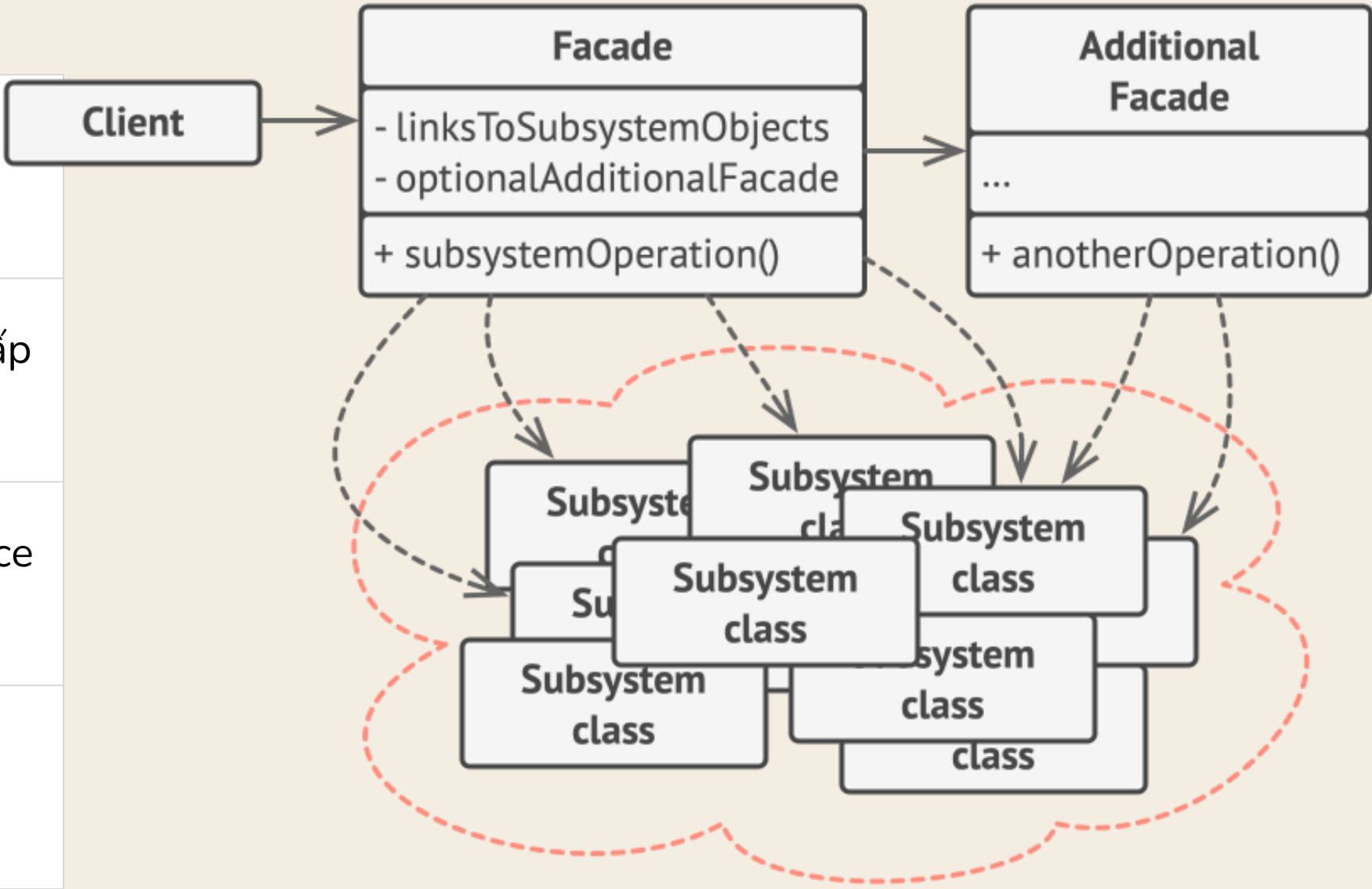
- Gom nhiều thao tác phức tạp của nhiều class con vào một class cấp cao, để client chỉ cần làm việc với một điểm truy cập đơn giản.

Facade thường làm các việc sau:

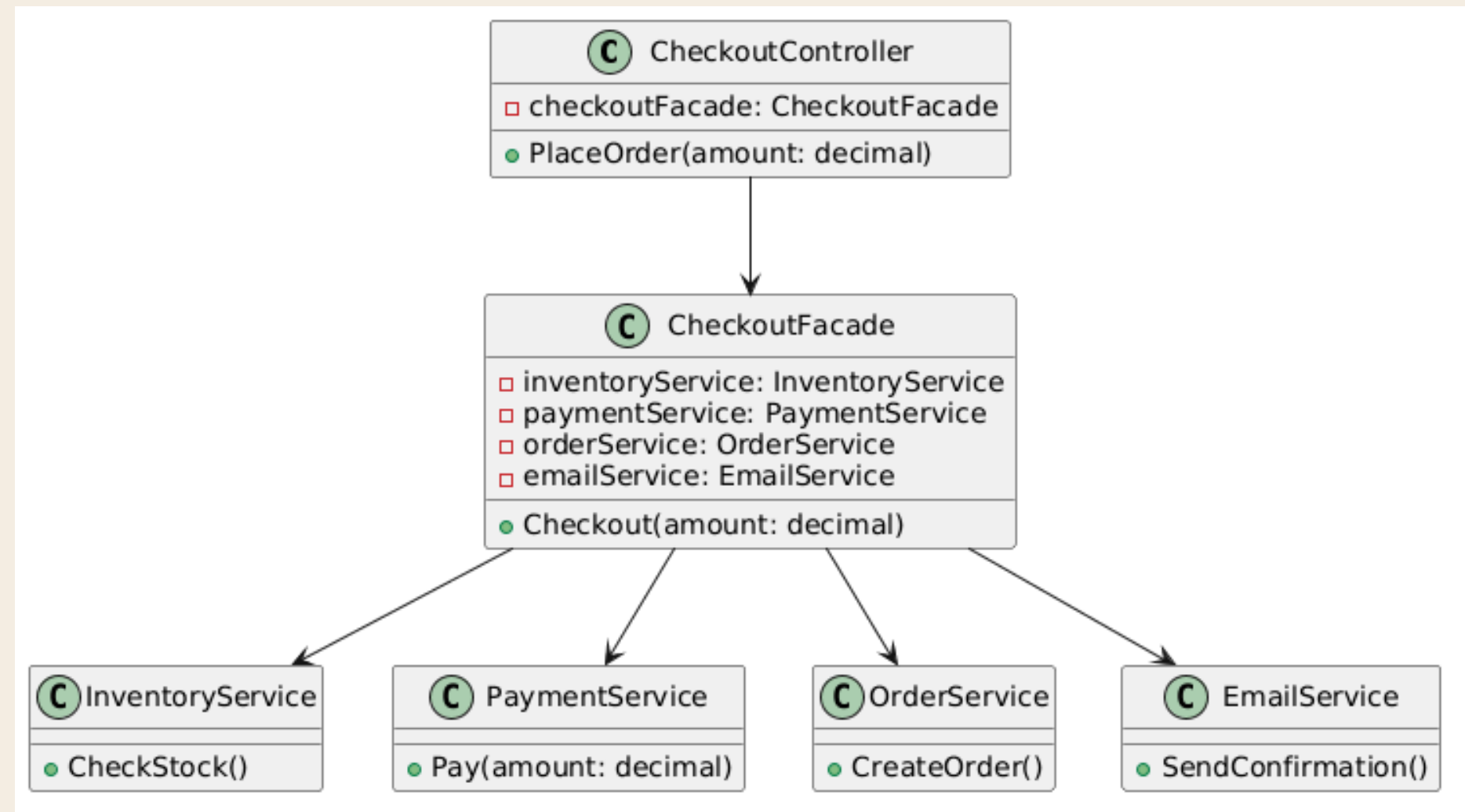
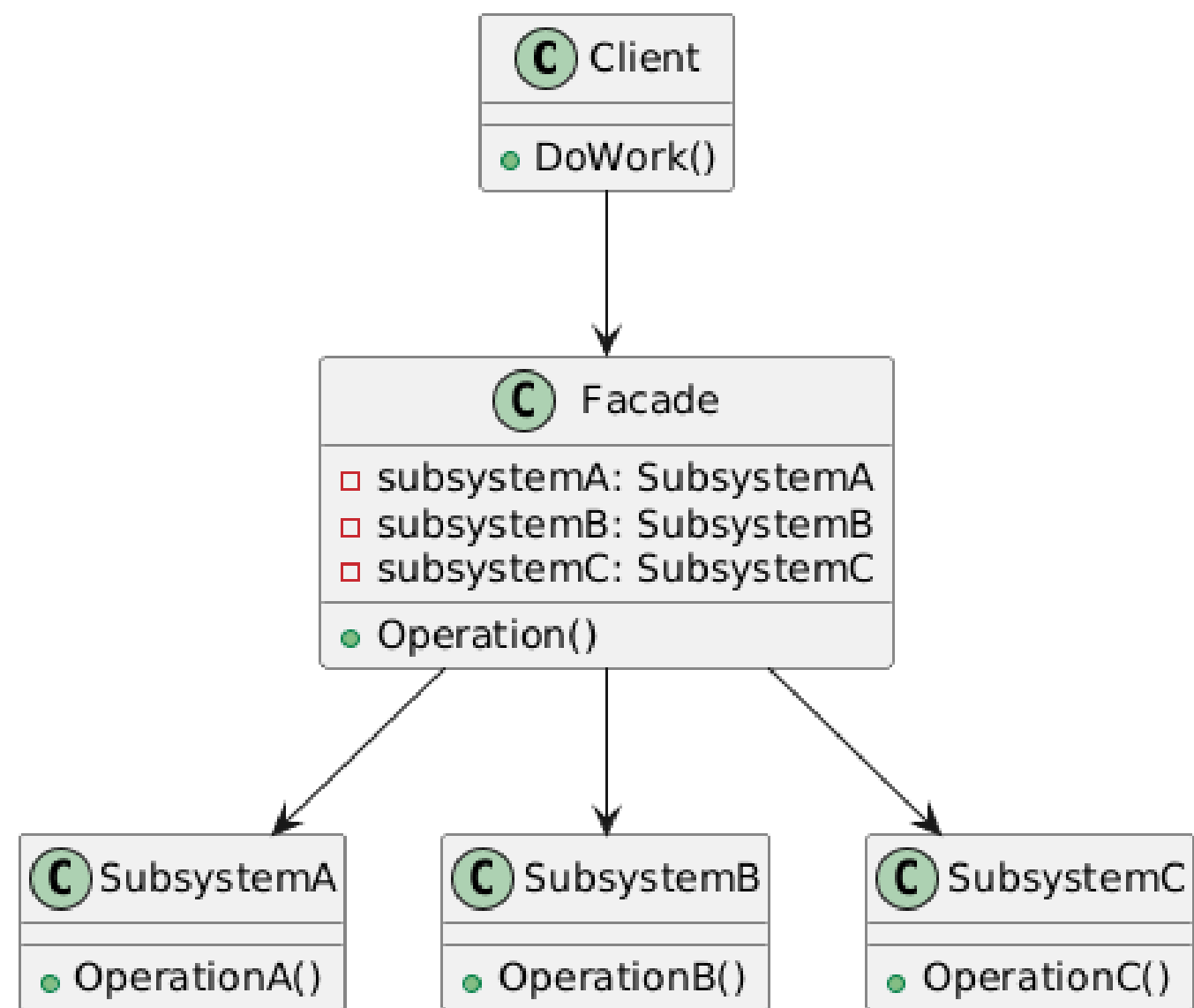
1. Nhận request từ client.
2. Gọi các subsystem cần thiết.
3. Điều phối thứ tự xử lý.
4. Ẩn chi tiết phức tạp bên trong.
5. Trả kết quả đơn giản cho client.

THÀNH PHẦN CHÍNH

Thành phần	Vai trò
Client	Đối tượng sử dụng chức năng cấp cao
Facade	Lớp trung gian cung cấp interface đơn giản
Subsystem Classes	Các class/service thật sự xử lý logic bên trong



UML



LUỒNG HOẠT ĐỘNG

Luồng hoạt động của Facade Pattern:

1. Client cần thực hiện một chức năng cấp cao.
2. Client gọi một phương thức đơn giản trên Facade.
3. Facade gọi các subsystem cần thiết.
4. Các subsystem xử lý công việc chuyên biệt của chúng.
5. Facade có thể tổng hợp kết quả.
6. Client nhận kết quả mà không cần biết chi tiết bên trong.

ĐÁNH GIÁ

Ưu điểm	Nhược điểm
Giảm độ phức tạp cho client	Facade có thể trở thành God Class
Che giấu chi tiết xử lý nội bộ	Có thể che giấu quá nhiều chi tiết quan trọng
Phù hợp với kiến trúc nhiều layer	Không thay thế được thiết kế subsystem tốt
Dễ bảo trì hơn khi subsystem thay đổi	Có thể làm giảm tính linh hoạt nếu client cần thao tác chi tiết

SO SÁNH

Facade vs Adapter

Tiêu chí	Facade	Adapter
Mục đích chính	Đơn giản hóa cách sử dụng subsystem phức tạp	Làm hai interface không tương thích làm việc với nhau
Vấn đề giải quyết	Client phải gọi quá nhiều class/phương thức	Client không dùng được class có sẵn vì interface không khớp
Số lượng class thường bọc	Nhiều class/subsystem	Thường một class hoặc một service
Có tập trung vào tương thích interface không?	Không phải trọng tâm chính	Có
Client dùng vì	Muốn API đơn giản hơn	Muốn dùng class không tương thích
Ví dụ	CheckoutFacade.Checkout() gọi nhiều service	StripeAdapter.Pay() gọi StripeService.MakePayment()

SO SÁNH

Adapter & Decorator

Tiêu chí	Adapter	Decorator
Mục đích chính	Làm interface không tương thích trở nên tương thích	Thêm chức năng mới cho object
Interface sau khi bọc	Thường khác với Adaptee	Thường giống object gốc
Có thay đổi hành vi không?	Có thể chuyển đổi lời gọi	Có, bằng cách bổ sung hành vi
Tập trung vào	Tương thích interface	Mở rộng chức năng
Ví dụ	StripeAdapter biến StripeService thành IPaymentGateway	LoggingPaymentGateway thêm logging trước/sau khi thanh toán

SO SÁNH

Facade vs Mediator

Tiêu chí	Facade	Mediator
Mục đích chính	Cung cấp interface đơn giản cho subsystem	Điều phối giao tiếp giữa nhiều object
Quan hệ với các object	Facade đứng trước subsystem	Mediator đứng giữa các object ngang hàng
Client có biết subsystem không?	Thường không cần biết	Các object thường biết mediator
Trọng tâm	Đơn giản hóa cách sử dụng	Giảm phụ thuộc nhiều-nhiều giữa các object
Ví dụ	CheckoutFacade gọi Payment, Inventory, Email	ChatRoomMediator điều phối các user gửi tin nhắn

Tháng 3 2026

THANK YOU

for Listening

Trình bày bởi Nhóm 5